📘 ASP.NET Core MVC – Assignment Set

🎯 Objective

By completing these assignments, you will be able to:

- Create Controllers, Models, and Views
- Pass data using ViewBag/ViewData
- Perform CRUD operations using EF Core
- Implement Model Validation
- Work with Forms using HTML Helpers & Tag Helpers

🧩 Assignment 1: Controllers (Basics)

📌 Tasks

1. Create a controller named **StudentController**
2. Add the following action methods:
    - Index() → returns a string "Welcome to Student Page"
    - Details() → returns "Student Details Page"
3. Add a parameterized action:
4. public IActionResult GetStudent(int id)

Display the ID in the output.

🎯 Expected Output

- URL: /Student/GetStudent/5
- Output: Student ID is 5

🧩 Assignment 2: Models (Data Structure)

📌 Tasks

1. Create a Model class **Student**
2. Add properties:
    - Id (int)
    - Name (string)
    - Age (int)
    - Email (string)
3. Create another model **Course**
    - CourseId
    - CourseName
4. Add a relationship:
    - One Student can have multiple Courses

🎯 **Expected Output**

- Proper model classes with navigation properties

🧩 **Assignment 3: Views (Razor Basics)**

📌 **Tasks**

1. Create a View for Index action
2. Pass a Student object from controller to view
3. Display:
    - Name
    - Age
    - Email
4. Use Razor syntax:
5. @Model.Name

🎯 **Expected Output**

- Student details displayed in browser

🧩 Assignment 4: ViewBag and ViewData

📌 Tasks

1. In Controller:
    - Store student name using **ViewBag**
    - Store student age using **ViewData**
2. Display both values in the View
3. Compare:
    - Which one needs casting?
    - Which one is dynamic?

🎯 Expected Output

- Values displayed correctly using both methods

🧩 Assignment 5: Model CRUD (Core Assignment)

📌 Scenario

Build a **Student Management System**

📌 Tasks

1. Setup:
    - Create DbContext
    - Configure SQL Server connection
    - Add DbSet
2. Implement CRUD:
    - Create → Add new student
    - Read → Display student list
    - Update → Edit student details

    - Delete → Remove student
3. Create corresponding Views:
    - Index
    - Create
    - Edit
    - Delete

🎯 **Expected Output**

- Fully working CRUD application

🧩 **Assignment 6: Model Validations**

📌 **Tasks**

Add Data Annotations to Student model:

```
[Required]
[StringLength(50)]
public string Name { get; set; }

[Range(18, 60)]
public int Age { get; set; }

[EmailAddress]
public string Email { get; set; }
```

1. Show validation messages in UI
2. Prevent form submission if invalid

🎯 **Expected Output**

- Validation errors displayed on form

🧩 Assignment 7: Forms using HTML Helpers

📌 Tasks

1. Create a form using HTML Helpers:
2. @using (Html.BeginForm())
3. {
4. @Html.TextBoxFor(m => m.Name)
5. @Html.ValidationMessageFor(m => m.Name)
6. }
7. Add fields:
    - Name
    - Age
    - Email
8. Submit form to controller

🎯 Expected Output

- Form submits data correctly to controller

🧩 Assignment 8: Forms using Tag Helpers

📌 Tasks

1. Create form using Tag Helpers:
2. <form asp-action="Create">
3. <input asp-for="Name" />
4.
5. </form>
6. Use:
    - asp-for
    - asp-action

    - asp-controller
7. Enable validation scripts

🎯 **Expected Output**

- Clean form with automatic binding and validation

🧩 **Assignment 9: Mini Project (Final Integration)**

📌 **Build:**

👉 **Student Registration Portal**

📌 **Requirements**

- Use MVC architecture
- Perform CRUD operations
- Apply validation
- Use Tag Helpers
- Use ViewBag/ViewData (at least once)
- Display success message after submission